A Comparative Study of Knowledge Graph-to-Text Generation Architectures in the Context of Conversational Agents

Hussam Ghanem[1,2] and Christophe Cruz[1]

[1] ICB, UMR 6306, CNRS, Unversité de Bourgogne
[2] Davi The Humanizers, Puteaux, France

Abstract. In this work, we delve into the dynamic landscape of Knowledge Graph-to-text generation, where structured knowledge graphs are transformed into coherent natural language text. Three key architectural paradigms are explored: Graph Neural Networks (GNNs), Graph Transformers (GTs), and linearization with sequence-to-sequence models. We discuss the advantages and limitations of these architectures, and we do some experiments on these architectures. Performance evaluations on WebNLG V.2 demonstrate the superiority of sequence-to-sequence Transformer-based models, especially when enriched with structural information from the graph. Despite being unsupervised, the CycleGT model also outperforms GNNs and GTs. However, practical constraints, such as computational efficiency and model validity, make sequence-to-sequence models the preferred choice for real-time conversational agents. Future research directions include enhancing the efficiency of GNNs and GTs, addressing scalability issues, handling multimodal knowledge graphs, improving interpretability, and devising data labeling strategies for domain-specific models. Cross-lingual and multilingual extensions can further broaden the applicability of these models in diverse linguistic contexts. In conclusion, the choice of architecture should align with specific task requirements and application constraints, and the field offers promising prospects for continued innovation and refinement.

Keywords: Conversational Agents, Knowledge Graphs, Natural Language Generation, Graph Neural Networks, Graph Transformers, Sequence-to-Sequence Models.

1 Introduction

Conversational agents, commonly known as chatbots, have emerged as sophisticated computer programs that emulate human-like conversations with users [60]. These agents find applications across various platforms, such as messaging services, mobile apps, and websites, enabling instantaneous customer support and handling routine tasks like answering queries and assisting with bookings. The ones incorporating knowledge graphs (KG) [38] and [21] have emerged prominently within the broad spectrum of architectural options. These conversational agents, rooted in KG, leverage the organized information within a knowledge graph to craft responses that closely mimic human language. These agents can access and employ structured information during conversations by utilizing the richly interconnected representation of

entities and their relationships within the knowledge graph, resulting in more accurate and comprehensive user interactions. Incorporating knowledge graphs in conversational agents significantly augments their capabilities, making interactions more informative and beneficial.

In the realm of natural language processing and knowledge representation, the survey on graph-to-text generation architectures assumes a vital role. By presenting a comprehensive overview of current techniques, this review guides researchers, practitioners, and decision-makers through this rapidly evolving landscape. Its insights into emerging trends, approach strengths, and limitations facilitate the design of effective systems for transforming structured data into coherent human-readable narratives. Furthermore, the survey encourages collaboration, knowledge sharing, and innovation within the field, thus advancing graph-to-text generation and bridging the gap between structured knowledge and natural language expression.

Recently, neural approaches have demonstrated remarkable performance surpassing traditional methods in achieving linguistic coherence. However, challenges persist in maintaining semantic consistency, particularly with long texts [44]. The inherent complexity of neural approaches also poses limitations, as they need more parameterization and control over the structure of the generated text. Consequently, while current neural approaches tend to lag template-based methods in terms of semantic consistency [45], they outperform them significantly in terms of linguistic coherence. This distinction can be attributed to the ability of large language models (LLM) to capture certain syntactic and semantic properties of the language. Despite the advantages neural approaches offer in terms of linguistic consistency, their performance in maintaining semantic consistency is still a work in progress.

Graph-to-text (G2T) generation is a natural language processing (NLP) task that involves transforming structured data from a graph format into human-readable text (Figure 1). In this task, a knowledge graph (a type of structured data representation where entities are nodes, and relationships between entities are edges) is converted into coherent sentences or paragraphs in a natural language. Generating text from graphs necessitates sophisticated methods in graph processing for extracting pertinent information and in natural language generation to produce coherent and contextually fitting text. This is a demanding yet valuable endeavor within the larger framework of content creation and communication driven by data.

In the natural language generation (NLG), two criteria [13] are used to assess the quality of the produced answers. The first criterion is semantic consistency (Semantic Fidelity) which quantifies the fidelity of the data produced against the input data. The most common indicators are 1/ Hallucination: It is manifested by the presence of information (facts) in the generated text that is not present in the input data; 2/ Omission: It is exemplified by the omission of one of the pieces of information (facts) in the generated text; 3/ Redundancy: This is manifested by the repetition of information in the generated text; 4/ Accuracy: The lack of accuracy is manifested by the modification of information such as the inversion of the subject and the direct object complement in the generated text; 5/ Ordering: It occurs when the sequence of information is different from the input data. The second criterion is linguistic coherence (Output Fluency) to evaluate the fluidity of the text and the linguistic constructions of the generated text, the segmentation of the text into different sentences, the use of anaphoric pronouns to reference entities and to have linguistically correct Sentences.

Our objective is to delve into the intricacies of deep neural network architectures that harness the power of graphs, aiming to gain a comprehensive understanding of their inherent strengths and limitations for optimal utilization in the context of conversational agents.

This paper follows a structured progression to delve into the landscape of knowledge graph-to-text (KG2T) generation. Beginning with an in-depth review of advanced KG2T approaches in Section 2, the paper proceeds to examine the architectures and innovations in Section 3. Section 4 explores the empirical aspects, encompassing model performance assessment, datasets, metrics, and experiments. Section 5 then encapsulates the findings and discussions, presenting the culmination of results. Finally, the paper concludes in Section 6 by critically evaluating the implications of the discussed techniques within the context of conversational agents.

2 Background

The goal of KG-to-text generation is to create comprehensible sentences in natural language based on knowledge graphs (KGs), all while upholding semantic coherence with the KG triplets (Fig. 1). The term "knowledge graph" has been in existence since 1972, but its current definition can be attributed to Google's introduction of its Knowledge Graph in 2012 [5]. This marked the beginning of a trend, with numerous companies like Airbnb, Amazon, eBay, Facebook, IBM, LinkedIn, Microsoft, and Uber also making similar announcements. This collective push has led to the widespread adoption of knowledge graphs across various industries [21]. Consequently, academic research in this domain has experienced a notable upsurge in recent years, resulting in a growing number of scholarly publications focused on knowledge graphs [21]. These graphs employ a data model based on graphs to efficiently manage, integrate, and extract valuable insights from large and diverse datasets [41].

The problem formulation is the following. Given the input KG G, which is composed of $\{(\mathrm{h}_1, r_1, t_1), \dots, (\mathrm{h}_n, r_n, t_n) | \ \mathrm{h}_*, \ t_* \in \mathrm{E}, \ \ r_* \in \ \mathrm{R}\}\}$, where E denotes the entity set and R represents the relation set, The objective of the KG-to-text generation task is to produce a coherent and logically sound sequence of text $T =< \mathrm{t}_1, \mathrm{t}_2, \dots, \mathrm{t}_n >, \mathrm{t}_k \in$ V where V denotes the vocabulary composed of n output tokens.

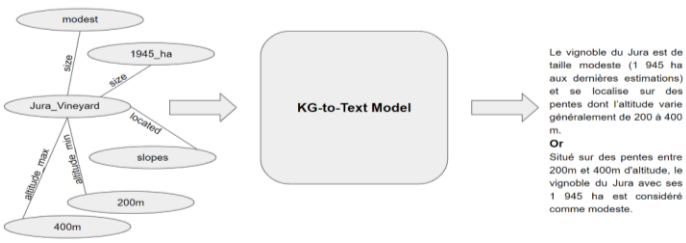

Fig. 1. a knowledge graph and two examples of its corresponding text using a KG-to-text generation model

Unlike the conventional text generation task (Seq2Seq), generating text from a knowledge graph adds the extra challenge of ensuring the accuracy of the words within the generated sentences. The current methods can be classified into three distinct

categories, as illustrated in (Fig. 2). We will later delve into these categories in more comprehensive detail in the section 3:

1. **Linearization** with Sequence-to-Sequence (Seq2Seq): This involves converting the graph G into a sequence G_{linear}=<g_1, g_2, …, g_m> consisting in of m input tokens for the sequence-to-sequence model.
1. **Graph Neural Networks** (GNNs) [50]: These models encode the topological structures of a graph and learn entity representations by aggregating the features of entities and their neighbors. GNNs are not standalone; they require a decoder to complete the encoder-decoder architecture.
2. **Graph Transformer** (GT): This is an enhanced version of the original transformer [57] model adapted to handle graph data.

Graph-to-Text (G2T) leverages both, graph embedding techniques and Pre-trained Language Models (PLMs). Graph embeddings and PLMs are both important for distinct reasons, playing crucial roles in G2T tasks. Graph embeddings allow us to capture subtle relationships between entities and properties in a numerical format, facilitating the manipulation of this data and the creation of generative models. The task of generating text and establishing alignments between source entities/relationships and target tokens is challenging for standard language models due to limited parallel graph-text data availability. The following two sections deal with these topics.

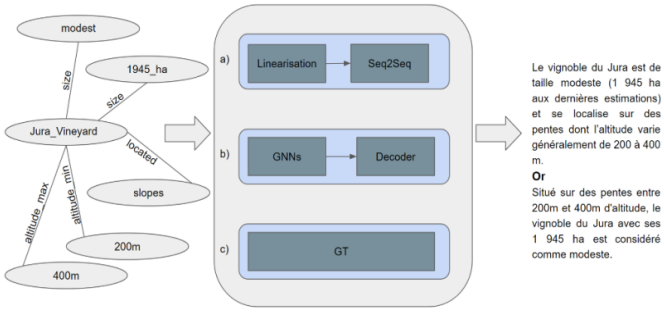

Fig. 2. The architecture of KG-to-text generation with the three categories of representation, a) Linearization + Seq2Seq, b) GNNs with decoder (e.g. LSTM), and c) Graph Transformer (GT)

2.1 Graph Embeddings

Graph embeddings in Graph Neural Networks (GNNs) refer to low-dimensional vectors assigned to nodes or subgraphs [9]. The goal is to create compact representations for downstream tasks, capturing both local node info and global graph structure. This involves iterative message passing, aggregating info between connected nodes to learn meaningful embeddings. The quality of these embeddings is vital for GNN performance and broader applications with graph data. For instance, 1/ Distance-based scoring functions underpin Translational Models in Knowledge Graph (KG) embeddings. TransE [3] assumes head entity + relation vector ≈ tail entity vector. TransH [61] introduces relation-specific hyperplanes, TransR [33] uses separate embeddings for entities and relations. 2/ A significant contribution [11] used message passing for insights from molecular graphs, pioneering differentiable architectures for graphs, like Message Passing Neural Network (MPNN) [16]. Nodes have hidden states, aggregated with adjacent nodes' states to update the node's state. 3/ Graph embedding methods

apply to knowledge graphs. ERNIE [64] and KnowBERT [43] use techniques like TransE [3] and TuckER [1] for link prediction, offering baseline for knowledge graph encoding [56].

2.2 Mainstream Architectures

Research focuses on creating generalist pre-trained language models for KG-to-text generation. A common method involves converting input graphs into text sequences and fine-tuning pre-trained seq2seq models. These models convert sequences from one domain to another using an encoder-decoder architecture. The encoder captures input sequence context, transmitting it to the decoder for output sequence generation. Both components can use RNNs, LSTMs, GRUs, Transformers, or similar structures.

Encoder-decoder architecture comprises two key parts: an encoder generating a fixed-length hidden representation from input and a decoder utilizing this to generate output. It suits scenarios with sequence inputs and outputs, like machine translation. While seq2seq is for variable-length sequences, encoder-decoder applies to broader data transformation tasks, excelling in sequence-to-sequence modeling.

Pre-trained models like BART [28], T5 [47], GPT [36] based on KG-to-text datasets [48, 24] perform well with fine-tuning due to self-supervised pre-training on vast unlabeled text corpora. They outperform complex models in KG-to-text tasks. This approach's details are in section 3.

Encoder modification replaces traditional sequential encoders in Graph2Seq models with structure-aware graph encoders like GCNs and graph-state LSTMs [26, 54] to improve KG-to-text generation. This shift enhances encoding of input structural data and surpasses standard Seq2Seq models in specific tasks. However, this change introduces a challenge: a structural gap between the encoder, processing graph data, and the decoder, generating linear text. This gap complicates aligning source and target information. To address this, researchers develop more sophisticated encoder structures, using techniques like GNNs and GTs to retain structural information from linearized graph inputs [15, 10, 39]. [66] propose a central "planner" component which organizes and generates coherent text from input KG, determining text order, style, and structure. By bridging the gap between structured KG data and natural language text, the planner contributes to fluent and coherent text generation.

Cycle training, both graph-to-text (G2T) and text-to-graph (T2G), faces a challenge due to limited training data availability. Deep learning models in these domains suffer due to scarce supervised samples compared to tasks like machine translation. To address this, cycle training techniques simultaneously learn conversions between G2T and T2G using non-parallel graph-to-text data [34, 17, 18, 51]. This approach enables direct output-input comparison for evaluation. However, traditional cycle-consistent training encounters issues with many-to-one or surjective mappings, leading to errors. An approach [18] uses a conditional variational autoencoder (CVAE) to address this, converting surjective mappings into implicit bijections, enhancing diversity and minimizing errors in both directions. Another strategy, the Deep ReAder-Writer (DRAW) network [34], includes Reader, Writer, and Reviewer modules. The Reader predicts new links in knowledge graphs (KGs) by considering multi-hop neighborhoods, augmenting KGs. The Writer encodes KGs using graph neural networks (GNNs) to generate paragraphs. The Reviewer module refines paragraph quality, addressing issues like word repetition and enhancing output diversity.

3 Knowledge Graph-to-text generation architectures

This section introduces three key components in KG-to-text generation task: Graph Linearization (GL), Graph Neural Networks (GNNs), and Graph Transformers (GTs). These elements collectively harness structured data within knowledge graphs. We'll discuss converting complex graphs into linear forms, GNNs' ability to capture relationships, and GTs' specialized handling of graph-based data. This analysis sheds light on their roles and significance across applications.

3.1 Graph Linearization

Graph linearization involves transforming intricate graph-based structures into linear or sequential formats. This rearrangement orders nodes and relationships, creating a sequence suitable for algorithms, tasks or models requiring linear input such as many methods in NLP. One approach involves linearizing the knowledge graph (KG) [48, 62, 15, 22] and using PLMs like GPT, BART, or T5 for seq2seq generation. PLMs can generalize for downstream NLG tasks [29], but most were trained on text data [46, 28], lacking structured input.

Heuristic search algorithms like breadth-first search (BFS) or predefined rules are common for graph linearization. However, these methods often neglect structural information during KG encoding, as they don't explicitly consider relationships between input entities.

Researches such as [39, 66, 19] introduce different neural planners to compute the input triples order before linearization. [39] employ data-driven scoring for precise plans, considering recommendations and user rules. Others use GCN-based neural planners [66] (Graph Convolution Network), reordering graph nodes for sequential content plans encoded by LSTM-based sequential encoders. Plan-and-pretrain techniques introduced by [19] leverage text planners based on relational graph convolutional networks (R-GCN) [66] and pretrained T5 Seq2Seq models. [25] proposed a joint graph-text learning framework called JointGT.

To address the issue when the input is a sequence of RDF triplets, [10] introduces a encoder model called GTR-LSTM. This model maintains the structure of RDF triplets within a small knowledge graph, enabling it to capture relationships both within individual triplets (intra-triple relations) and between interconnected triplets (inter-triple relations). This approach improves sentence generation accuracy. Unlike TreeLSTM [55], which lacks cycle handling, GTR-LSTM handles cycles by utilizing a combination of topological sorting and breadth-first traversal. An attention model is employed to gather comprehensive global information from the knowledge graph. Additionally, unlike GraphLSTM [31] that only supports specific entity relations, all relations and incorporates them into the calculation of hidden states [10].

In efforts to preserve the graph's topology, despite striving for maximum retention using seq2seq methods, the Transformer-based seq2seq models come with significant costs, particularly during the pretraining phase. Additionally, the computational expense of linearization can become substantial when dealing with extensive knowledge graphs. Therefore, to more effectively maintain the graph's topology, the introduction of Graph Neural Networks (GNNs) has been proposed, and their details will be explored in the following section.

3.2 Graph Neural Networks (GNNs)

Various approaches employ different versions of Graph Neural Network (GNN) architectures for processing graph-structured data. GNNs, including Graph Convolutional Networks (GCNs) [26], extended forms like Syn-GCNs [37] and DCGCNs [20], Graph Attention Networks (GATs) [59], and Gated Graph Neural Networks (GGNNs) [7, 6, 30], are well-suited for modeling entity relationships within knowledge graphs to generate text. In knowledge graph-to-text generation, GNNs have demonstrated promise by effectively representing relationships between entities in a knowledge graph.

GCNs, Syn-GCNs, and DCGCNs encode entity relationships into low-dimensional representations. GATs dynamically assign weights to entities and relationships to influence text generation. GGNNs utilize gating mechanisms to control information flow among graph nodes, aiding in context incorporation. Some studies combine GNNs with reinforcement learning to optimize text generation based on the knowledge graph [6].

Overall, the use of GNNs for knowledge graph to text generation is an active area of research, with many recent studies exploring different architectures and training methods. The results so far suggest that GNNs can effectively capture the relationships between entities in a knowledge graph and generate high-quality text based on that information. limitation relied to KG-to-Text generation with GNNs, is that GNNs can be computationally expensive and may struggle to handle large knowledge graphs. Additionally, their performance may degrade for graphs with complex relationships or structures. Despite these limitations, GNNs remain a promising direction for knowledge graph-to-text generation

3.3 Graphs Transformers (GTs)

In order to benefit from the power of models based on Transformer and to be able to model tree or graph-type data structures as with GNNs, and to overcome the limitations of local neighborhood aggregation while avoiding strict structural inductive biases, recent works have proposed to adapt the Transformer architecture. As Graph Transformers are equipped with self-attention mechanisms, they can capture global context information by applying these mechanisms to the nodes of the graph.

According to [4], GT differs from GNNs in that it allows direct modeling of dependencies between any pair of nodes regardless of their distance in the input graph. An undesirable consequence is that it essentially treats any graph as a fully connected graph, greatly reducing the explicit structure of the graph. To maintain a structure-aware view of the graph, their proposed model introduces an explicit relationship encoding and integrates it into the pairwise attention score computation as a dynamic parameter.

From the GNNs pipeline, if we make several parallel heads of neighborhood aggregation and replace the sum on the neighbors by the attention mechanism, e.g. a weighted sum, we would get the Graph Attention Network (GAT). Adding normalization and MLP feed-forward, we have a Graph Transformer [23]. For the same reasons as Graph Transformer, [35] presents the K-BERT model, they introduce four components to augment the Transformer architecture and to be able to handle a graph as input. The first knowledge layer component takes a sentence and a set of triplets as input and outputs the sentence tree by expanding the sentence entities with their cor-

responding triplets. They also add the Seeing Layer component to preserve the original sentence structure and model the relationships of the triples by building a Visibility Matrix. Another component is the Mask-Transformer where they modify the self-attention layer to consider the visibility matrix when calculating attention.

The use of Graph Transformers for Knowledge graph to text generation has gained popularity in recent years [52, 27] due to their ability to effectively handle graph structures and capture the relationships between nodes in the graph. Additionally, Graph Transformers can handle large graphs and have the ability to model long-range dependencies between nodes in the graph. Despite the advantages, the training of Graph Transformers can be computationally expensive and the interpretability of the model remains a challenge. Overall, the use of Graph Transformers for Knowledge graph to text generation is a promising area of research and can lead to significant improvements in the generation of text from knowledge graphs.

4 Experiments on some models, Datasets and metrics used

4.1 Datasets and metrics

we turn our attention to the datasets utilized in the implementation of various models. During the training and validation phases, the utilized datasets align with those employed by each specific model [53, 40, 15, 12] (Table 1). However, when transitioning to the testing and evaluation phase, a significant portion of the models is subjected to assessment using the Webnlg v.2 dataset [53], thereby providing a standardized and consistent benchmark for performance evaluation. In light of the current challenges surrounding the availability of a Webnlg v.2 dataset tailored for CycleGT [17] model and P2 [19] model, our evaluation approach diverges. Instead, we conduct the evaluation of these models on the test dataset corresponding to the same version as the utilized training and validation datasets. These approaches are necessitated by the absence of a dedicated Webnlg v.2 dataset compatible with these models at this juncture.

Table 1. Datasets used in our experiments

Dataset	Train	Valisdation	Test
WebNLG V.2 2018 [53]	34.3k	4.3k	4.2k
WebNLG 2017 [15]	18.1k	0.87k	1.8k
WebNLG 2020 [12]	35.4k	4.4k	5.1k
DART [40]	62.6k	6.9k	12.5k

In our conducted experiments, we employed a set of ngram-based metrics, namely Rouge [32], Meteor [2], Bleu [42], and Cider [58], to assess the effectiveness of our developed model. The selection of these four distinct metrics aims to comprehensively analyze various aspects of the evaluated text. These metrics encompass precision (Bleu), recall (Rouge), F-score (Bleu and Rouge), as well as the consideration of term frequency-inverse document frequency (TF-Idf) aspects (Cider). This approach enables a comprehensive evaluation of our model's performance across multiple linguistic dimensions.

4.2 Experiments

We conducted an empirical assessment of several approaches, executed on our local machines and evaluated using the same test dataset. Our evaluation encompassed various types of approaches, including graph linearisation [25, 43, 19], a model employing Graph Neural Networks (GNNs) [49], and another employing Graph Transformers (GTs) [52]. Additionally, we implemented an unsupervised cycling model [17]. While preserving most of the original characteristics of these models, we occasionally adjusted hyperparameters for resource limitations or faster training.

Used graph linearisation approaches. Here we present models that use PLMs as seq2seq models to do KG2T generation.

In the "JointGT" approach [25], T5 and BART served as foundational models. These models underwent a pretraining phase on the KGTEXT dataset [8] and were further refined through tasks involving text and graph reconstruction as well as alignment prediction. This comprehensive process resulted in the creation of two pretrained models known as JointGT_T5 and JointGT_BART. In our implementation, we chose to work with JointGT_T5, maintaining the original fine-tuning hyperparameters while making a batch size adjustment to 16 during both training and prediction phases to accommodate computational limitations.

The adaptation of BART and T5 for graph-to-text generation by adding <H>, <R>, and <T> tokens to the models' vocabulary [48] led to vocabulary extension for KG datasets and fine-tuning on WebNLG 2017 and augmented dataset DART [40]. We maintained their implementation with slight modifications, increasing the batch size to 32 for faster fine-tuning.

[19] employed the T5-Large model and the planner module using DGL, Pytorch, and Transformers. Most hyperparameters were retained, with the batch size adjusted to 1 due to resource limitations.

Used GNNs approaches. [49] implemented their models using PyTorch Geometric (PyG) and OpenNMT-py, employing byte pair encoding (BPE) for entity word segmentation. Our implementation maintained their hyperparameters, utilizing cge-lw (Parallel Graph Encoder - layer-wise) as the graph encoder where global and local node representations are concatenated layer-wise.

Used GTs approaches. Graformer [52] introduced additional preprocessing steps, including training a BPE vocabulary and incorporating specialized tags for entities and relations. Their training implemented an Epoch curriculum strategy. Our implementation mirrored their hyperparameters.

Used unsupervised cycling approaches. The unsupervised CycleGT model [17] introduced a non-parallel training and validation dataset by segregating text and graph data. In our implementation, we adhered to CycleGT's GitHub repository specifications.

Overall, our experiments provide comprehensive insights into the behaviors and performance of these diverse approaches within the context of graph-to-text generation, and we discuss the results (Table 1) in the next section.

5 Results and Discussion

Each architecture has its own advantages and disadvantages, and the choice of architecture will depend on the specific requirements of the actual task.

In light of these elements, and with the constraint of the data labialization for specific domains of KG2T generation, we choose to go further with seq2seq Transformer based models (PLMs) in our Knowledge Graph-to-Text Generation.

We see in Table 2 that all models using seq2seq Transformer based models have better results than GNNs and GTs models, and with big margin when these models have some additional processes before linearization to keep some information about the structure of the graph like in JointGT [25] and P2 [19]. Even CycleGT model has results better than GNNs and GTs models, which is an unsupervised model and which uses a linearization phase on the graph.

As mentioned before, we evaluate our models on WebNLG V.2 [53] which has 1600 test instance (source KGs), except P2 and CycleGT, which are evaluated on enriched version of WebNLG V.2 [53] and which has 1860 test instance.

6 Conclusion and perspectives

In this paper, we have explored and compared three distinct architectures for Knowledge Graph-to-text generation: Graph Neural Networks (GNNs), Graph Transformers (GTs), and linearization with seq2seq models, primarily Pre-trained Language Models (PLMs). Each of these architectures has its unique advantages and limitations, making them suitable for different scenarios and use cases.

GNNs offer a flexible and scalable approach to model graph structures and relationships, but they may struggle with efficiency when handling large and complex knowledge graphs. GTs, on the other hand, provide a specialized solution for graph-based tasks, offering a more direct and efficient way to process graph structures. However, they may require extensive training data and computational resources.

Linearization with seq2seq models, especially those based on PLMs, simplifies the process by converting knowledge graphs into linear sequences and generating text from them. This approach, despite its simplicity, can lose some structural information during linearization. However, as shown in our experiments, seq2seq Transformer-based models consistently outperformed GNNs and GTs, espacially when models incorporate additional processes to retain graph structure information like JointGT and P2.

In the context of conversational agents, where factors like response time and correctness are critical, the inference time and resource requirements of GNNs and GTs can be limiting. Hence, for practical deployment, seq2seq Transformer-based models stand out as a more feasible choice, given their superior performance and efficiency.

Looking ahead, the field of Knowledge Graph-to-text generation presents several avenues for advancement. Firstly, there's a pressing need to enhance the computational efficiency of Graph Neural Networks (GNNs) and Graph Transformers (GTs) to make them more suitable for real-time applications. This involves optimizing their architectures, parallelizing computations, and harnessing hardware accelerators. Additionally, as the scale and complexity of knowledge graphs continue to grow, develop-

ing strategies to effectively handle large graphs while maintaining performance is a significant challenge that warrants further exploration.

Moreover, the extension of these approaches to accommodate multimodal knowledge graphs, which integrate textual, visual, and other data types, could open up new horizons for comprehensive information retrieval and generation, especially with the use of Meta-tranformers multi-modal approach [65]. Furthermore, ensuring the interpretability of GNNs and GTs is crucial for building trust in generated text, particularly in domains like healthcare and law. Moreover, addressing the scarcity of labeled data for specific knowledge domains through innovative data labeling and augmentation techniques can enhance the training of domain-specific models. Lastly, the advancement of these models to handle multiple languages and facilitate cross-lingual knowledge transfer holds promise for their broader applicability in diverse linguistic contexts

In conclusion, the choice of architecture for Knowledge Graph-to-text generation should be guided by the specific requirements of the task and the constraints of the application. While GNNs and GTs offer valuable approaches for certain scenarios, the efficiency and performance of seq2seq Transformer-based models make them a compelling choice for many real-world applications. Future research should aim to address the challenges and opportunities presented by these diverse architectures to further advance the field.

Table 2. The performance of used the implemented models on WebNLG V.2 [53]

		Metric	**BLEU**				**METEOR**	**ROUGE_L**	**CIDEr**
		Model	**1**	**2**	**3**	**4**			
Graph Lineariza-tion	Supervised	PLMs-G2T (webnlg) [48]	79.18	67.66	57.31	48.54	40.28	64.91	3.34
		PLMs-G2T (DART) [48]	83.45	72.77	63.22	55.03	43.20	68.95	3.82
		P2_35 [19]	86.91	76.30	66.49	57.95	44.36	70.31	**4.98**
		JointGT [25]	**90.35**	**81.82**	**73.60**	**66.14**	**47.32**	**75.96**	4.59
	Unsupervised	CycleGT [17]	74.79	62.88	53.68	46.16	36.21	68.26	3.20
	GNNs	KG2T_G&L [49]	60.93	49.36	40.99	34.42	28.11	49.04	2.02
	GTs	Graformer [52]	70.67	59.72	51.12	44.22	33.82	58.13	2.79

7 Acknowledgement

The authors thank the French company DAVI (Davi The Humanizers, Puteaux, France) for their support, and the French government for the plan France Relance funding.

8 References

1. Balažević, Ivana, Carl Allen, and Timothy M. Hospedales. "Tucker: Tensor factorization for knowledge graph completion." arXiv preprint arXiv:1901.09590 (2019).

2. Banerjee, Satanjeev, and Alon Lavie. "METEOR: An automatic metric for MT evaluation with improved correlation with human judgments." Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization. 2005.
3. Bordes, Antoine, et al. "Translating embeddings for modeling multi-relational data." Advances in neural information processing systems 26 (2013).
4. Cai, Deng, and Wai Lam. "Graph transformer for graph-to-sequence learning." Proceedings of the AAAI conference on artificial intelligence. Vol. 34. No. 05. 2020.
5. Chaudhri, Vinay, et al. "Knowledge graphs: introduction, history and, perspectives." AI Magazine 43.1 (2022): 17-29.
6. Chen, Yu, Lingfei Wu, and Mohammed J. Zaki. "Reinforcement learning based graph-to-sequence model for natural question generation." arXiv preprint arXiv:1908.04942 (2019).
7. Chen, Yu, Lingfei Wu, and Mohammed J. Zaki. "Toward Subgraph-Guided Knowledge Graph Question Generation With Graph Neural Networks." *IEEE Transactions on Neural Networks and Learning Systems* (2023).
8. Chen, Wenhu, et al. "KGPT: Knowledge-grounded pre-training for data-to-text generation." *arXiv preprint arXiv:2010.02307* (2020).
9. Dai, Yuanfei, et al. "A survey on knowledge graph embedding: Approaches, applications and benchmarks." Electronics 9.5 (2020): 750.
10. Distiawan, Bayu, et al. "GTR-LSTM: A triple encoder for sentence generation from RDF data." Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2018.
11. Duvenaud, David K., et al. "Convolutional networks on graphs for learning molecular fingerprints." Advances in neural information processing systems 28 (2015).
12. Ferreira, Thiago Castro, et al. "Enriching the WebNLG corpus." Proceedings of the 11th International Conference on Natural Language Generation. 2018.
13. Ferreira, Thiago Castro, et al. "Neural data-to-text generation: A comparison between pipeline and end-to-end architectures." arXiv preprint arXiv:1908.09022 (2019).
14. Fu, Zihao, et al. "Partially-aligned data-to-text generation with distant supervision." *arXiv preprint arXiv:2010.01268* (2020).
15. Gardent, Claire, et al. "The WebNLG challenge: Generating text from RDF data." Proceedings of the 10th International Conference on Natural Language Generation. 2017.
16. Gilmer, Justin, et al. "Neural message passing for quantum chemistry." International conference on machine learning. PMLR, 2017.
17. Guo, Qipeng, et al. "Cyclegt: Unsupervised graph-to-text and text-to-graph generation via cycle training." arXiv preprint arXiv:2006.04702 (2020).
18. Guo, Qipeng, et al. "Fork or fail: Cycle-consistent training with many-to-one mappings." International Conference on Artificial Intelligence and Statistics. PMLR, 2021.
19. Guo, Qipeng, et al. "P2: A Plan-and-Pretrain Approach for Knowledge Graph-to-Text Generation: A Plan-and-Pretrain Approach for Knowledge Graph-to-Text Generation." Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+). 2020.
20. Guo, Zhijiang, et al. "Densely connected graph convolutional networks for graph-to-sequence learning." Transactions of the Association for Computational Linguistics 7 (2019): 297-312.
21. Hogan, Aidan, et al. "Knowledge graphs." ACM Computing Surveys (Csur) 54.4 (2021): 1-37.
22. Hoyle, Alexander, Ana Marasović, and Noah Smith. "Promoting graph awareness in linearized graph-to-text generation." *arXiv preprint arXiv:2012.15793* (2020).
23. Joshi, Chaitanya. "Transformers are graph neural networks." The Gradient 7 (2020).
24. Kale, Mihir, and Abhinav Rastogi. "Text-to-text pre-training for data-to-text tasks." *arXiv preprint arXiv:2005.10433* (2020).

25. Ke, Pei, et al. "Jointgt: Graph-text joint representation learning for text generation from knowledge graphs." arXiv preprint arXiv:2106.10502 (2021).
26. Kipf, Thomas N., and Max Welling. "Semi-supervised classification with graph convolutional networks." arXiv preprint arXiv:1609.02907 (2016).
27. Koncel-Kedziorski, Rik, et al. "Text generation from knowledge graphs with graph transformers." arXiv preprint arXiv:1904.02342 (2019).
28. Lewis, Mike, et al. "Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension." arXiv preprint arXiv:1910.13461 (2019).
29. Li, Junyi, et al. "Pretrained language models for text generation: A survey." arXiv preprint arXiv:2201.05273 (2022).
30. Li, Yujia, et al. "Gated graph sequence neural networks." *arXiv preprint arXiv:1511.05493* (2015).
31. Liang, Xiaodan, et al. "Semantic object parsing with graph lstm." Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14. Springer International Publishing, 2016.
32. Lin, Chin-Yew. "Rouge: A package for automatic evaluation of summaries." Text summarization branches out. 2004.
33. Lin, Y., Liu, Z., Sun, M., Liu, Y., Zhu, X.: Learning entity and relation embeddings for knowledge graph completion. Proceedings of the AAAI conference on artificial intelligence 29(1) (2015)
34. Liu, Li, et al. "How to train your agent to read and write." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 35. No. 15. 2021.
35. Liu, Weijie, et al. "K-bert: Enabling language representation with knowledge graph." Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 34. No. 03. 2020.
36. Mager, Manuel, et al. "GPT-too: A language-model-first approach for AMR-to-text generation." arXiv preprint arXiv:2005.09123 (2020).
37. Marcheggiani, Diego, Anton Frolov, and Ivan Titov. "A simple and accurate syntax-agnostic neural model for dependency-based semantic role labeling." arXiv preprint arXiv:1701.02593 (2017).
38. Moon, Seungwhan, et al. "Opendialkg: Explainable conversational reasoning with attention-based walks over knowledge graphs." Proceedings of the 57th annual meeting of the association for computational linguistics. 2019.
39. Moryossef, Amit, Yoav Goldberg, and Ido Dagan. "Step-by-step: Separating planning from realization in neural data-to-text generation." arXiv preprint arXiv:1904.03396 (2019).
40. Nan, Linyong, et al. "Dart: Open-domain structured data record to text generation." arXiv preprint arXiv:2007.02871 (2020).
41. Noy, Natasha, et al. "Industry-scale Knowledge Graphs: Lessons and Challenges: Five diverse technology companies show how it's done." Queue 17.2 (2019): 48-75.
42. Papineni, Kishore, et al. "Bleu: a method for automatic evaluation of machine translation." Proceedings of the 40th annual meeting of the Association for Computational Linguistics. 2002.
43. Peters, Matthew E., et al. "Knowledge enhanced contextual word representations." arXiv preprint arXiv:1909.04164 (2019).
44. Puduppully, Ratish, Li Dong, and Mirella Lapata. "Data-to-text generation with content selection and planning." Proceedings of the AAAI conference on artificial intelligence. Vol. 33. No. 01. 2019.
45. Puzikov, Yevgeniy, and Iryna Gurevych. "E2e nlg challenge: Neural models vs. templates." Proceedings of the 11th International Conference on Natural Language Generation. 2018.
46. Radford, Alec, et al. "Language models are unsupervised multitask learners." *OpenAI blog* 1.8 (2019): 9.

47. Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." The Journal of Machine Learning Research 21.1 (2020): 5485-5551.
48. Ribeiro, Leonardo FR, et al. "Investigating pretrained language models for graph-to-text generation." arXiv preprint arXiv:2007.08426 (2020).
49. Ribeiro, Leonardo FR, et al. "Modeling global and local node contexts for text generation from knowledge graphs." Transactions of the Association for Computational Linguistics 8 (2020): 589-604.
50. Scarselli, Franco, et al. "The graph neural network model." IEEE transactions on neural networks 20.1 (2008): 61-80.
51. Schmitt, Martin, et al. "An unsupervised joint system for text generation from knowledge graphs and semantic parsing." *arXiv preprint arXiv:1904.09447* (2019).
52. Schmitt, Martin, et al. "Modeling graph structure via relative position for text generation from knowledge graphs." arXiv preprint arXiv:2006.09242 (2020).
53. Shimorina, Anastasia, and Claire Gardent. "Handling rare items in data-to-text generation." Proceedings of the 11th international conference on natural language generation. 2018.
54. Song, Linfeng, et al. "A graph-to-sequence model for AMR-to-text generation." arXiv preprint arXiv:1805.02473 (2018).
55. Tai, Kai Sheng, Richard Socher, and Christopher D. Manning. "Improved semantic representations from tree-structured long short-term memory networks." arXiv preprint arXiv:1503.00075 (2015).
56. Tucker, Ledyard R. "Some mathematical notes on three-mode factor analysis." Psychometrika 31.3 (1966): 279-311.
57. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... \& Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems, 30.
58. Vedantam, Ramakrishna, C. Lawrence Zitnick, and Devi Parikh. "Cider: Consensus-based image description evaluation." Proceedings of the IEEE conference on computer vision and pattern recognition. 2015.
59. Veličković, Petar, et al. "Graph attention networks." *arXiv preprint arXiv:1710.10903* (2017).
60. Wahde, Mattias, and Marco Virgolin. "Conversational agents: Theory and applications." HANDBOOK ON COMPUTER LEARNING AND INTELLIGENCE: Volume 2: Deep Learning, Intelligent Control and Evolutionary Computation. 2022. 497-544.
61. Wang, Z., Zhang, J., Feng, J., Chen, Z.: Knowledge Graph Embedding by Translating on Hyperplanes. In: Proceedings of the 28th AAAI Conference on Artificial Intelligence. pp. 1112–1119. AAAI Press (2014)
62. Yang, Zixiaofan, et al. "Improving text-to-text pre-trained models for the graph-to-text task." *Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+)*. 2020.
63. Yao, Shaowei, Tianming Wang, and Xiaojun Wan. "Heterogeneous graph transformer for graph-to-sequence learning." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020.
64. Zhang, Zhengyan, et al. "ERNIE: Enhanced language representation with informative entities." arXiv preprint arXiv:1905.07129 (2019).
65. Zhang, Yiyuan, et al. "Meta-transformer: A unified framework for multimodal learning." *arXiv preprint arXiv:2307.10802* (2023).
66. Zhao, Chao, Marilyn Walker, and Snigdha Chaturvedi. "Bridging the structural gap between encoding and decoding for data-to-text generation." Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics. 2020.